

Technical Documentation: Bubble Sort with Pass Matrix Storage

Author: Daksh Agarwal

Version: 1.0 | Date: January 23, 2026

Abstract

This document provides the technical specification for a C program that sorts integers from a file using the Bubble Sort algorithm. The program’s core functionality is to store the state of the array after each sorting pass into a 2D matrix for visualization.

Contents

1	System Overview	2
1.1	Purpose	2
1.2	High-Level Architecture	2
2	Component Specifications	2
2.1	Data Structures	2
2.1.1	int numbers[MAX_SIZE]	2
2.1.2	int passes_matrix[MAX_SIZE][MAX_SIZE]	2
2.2	Algorithm: Bubble Sort	2
2.2.1	Specification	2
2.2.2	Core Logic Snippet	3
3	Performance Analysis	3
3.1	Time Complexity	3
3.2	Space Complexity	3
4	Build and Execution Guide	3
4.1	Dependencies	3
4.2	Input File Specification	3
4.3	Compilation	4
4.4	Execution	4

1 System Overview

1.1 Purpose

The program is a command-line utility designed to read a list of integers from an input file, sort them in ascending order using Bubble Sort, and output a matrix visualizing the state of the array after each pass of the algorithm.

1.2 High-Level Architecture

The application is a monolithic program with a single execution thread. Its operation is sequential and can be broken down into four logical stages:

1. **File Ingestion:** Reads integer data from `input.txt`.
2. **Data Storage:** Populates a 1D array with the ingested data.
3. **Processing:** Executes the Bubble Sort algorithm and stores intermediate results in a 2D matrix.
4. **Output:** Prints the 2D matrix to standard output.

2 Component Specifications

2.1 Data Structures

2.1.1 `int numbers[MAX_SIZE]`

- **Type:** 1D Integer Array.
- **Purpose:** The primary working array. It holds the integers read from the file and is sorted in-place.
- **Capacity:** `MAX_SIZE`, defined as 100. The actual number of elements used is tracked by the variable `n`.

2.1.2 `int passes_matrix[MAX_SIZE][MAX_SIZE]`

- **Type:** 2D Integer Array (Matrix).
- **Purpose:** Serves as a log or history of the sorting process. Row `i` of the matrix stores a snapshot of the `numbers` array after the completion of pass `i` of the Bubble Sort algorithm.
- **Capacity:** `MAX_SIZE` x `MAX_SIZE`.

2.2 Algorithm: Bubble Sort

2.2.1 Specification

The program implements a non-optimized variant of the Bubble Sort algorithm.

- **Outer Loop (Pass Control):** Iterates a fixed `n` times, from `i = 0` to `n-1`.
- **Inner Loop (Comparison/Swap):** Iterates from `j = 0` to `n-2`. It compares `array[j]` and `array[j+1]` and swaps them if they are in the wrong order.
- **Design Note:** The algorithm deliberately omits optimizations (like early termination or a shrinking inner loop boundary) to ensure that exactly `n` passes are executed and stored, resulting in a consistent `n` x `n` output matrix for visualization.

2.2.2 Core Logic Snippet

```
1 // Outer loop controls the n passes for n snapshots
2 for (i = 0; i < n; i++) {
3     // Inner loop performs the pair-wise comparisons
4     for (j = 0; j < n - 1; j++) {
5         if (numbers[j] > numbers[j + 1]) {
6             // Standard swap logic
7             temp = numbers[j];
8             numbers[j] = numbers[j + 1];
9             numbers[j + 1] = temp;
10        }
11    }
12    // After a pass, copy the current array state to the matrix
13    for (k = 0; k < n; k++) {
14        passes_matrix[i][k] = numbers[k];
15    }
16 }
```

Listing 1: The core sorting and state-capture loop.

3 Performance Analysis

3.1 Time Complexity

The runtime performance is dominated by the nested loops of the sorting algorithm.

- **Complexity:** $O(n^2)$
- **Justification:** The outer loop runs n times and the inner loop runs $n-1$ times, leading to $n \times (n-1)$ comparison operations.
- **Cases:** Best, average, and worst-case time complexities are all $O(n^2)$ due to the non-optimized implementation.

3.2 Space Complexity

The memory footprint is determined by the storage required for the arrays.

- **Complexity:** $O(n^2)$
- **Justification:** While the primary working array requires $O(n)$ space, the `passes_matrix` requires $n \times n$ storage cells, making it the dominant factor.

4 Build and Execution Guide

4.1 Dependencies

- A standard C compiler (e.g., GCC, Clang).
- A standard C library.

4.2 Input File Specification

- **Filename:** The program hardcodes the input filename as `input.txt`.
- **Format:** The file must contain space-separated or newline-separated integers. Non-integer characters will terminate the reading process.

4.3 Compilation

The program can be compiled using the following standard GCC command:

```
gcc program_name.c -o bubble_sort_app
```

4.4 Execution

To run the program, execute the compiled binary from the command line. Ensure that `input.txt` is present in the same directory.

```
./bubble_sort_app
```